OurTearoa

INF803 - Cloud Computing Project
Cloud Web Application Development and Deployment

Submission Date: 16 October 2026

Mathias Jørgensen
Student ID: 26344066
Auckland University of Technology
cvm6264@autuni.ac.nz

Tim Schiller
Student ID: 26347002
Auckland University of Technology
zpn6423@autuni.ac.nz



Preamble

Project Deliverables & Access Links

Presentation Video	https://youtu.be/RNysyNhTvGI
GitHub Repository	https://github.com/bundgaard1/OurTearoa
Live Web Client URL	https://ourtearoa.duckdns.org
Live REST API Base	https://ourtearoa.duckdns.org/api

Member Contributions

Member Name	Student ID	Assigned Components & Cloud Tasks
Mathias Jorgensen	26344066	Setup AWS (S3 storage, EC2 compute, RDS PostgreSQL), Project Owner, Backend API Development (Express/Prisma), S3 asset integration
Tim Schiller	26347002	Frontend SPA Development (Angular 22, Signals), Project Manager, JWT Auth Architecture, Nginx/Certbot TLS setup, GitHub Actions CI/CD Pipeline

1. Introduction

1.1. Project Overview and Domain

OurTearoa is a cloud-native travel exploration platform engineered to streamline how users discover and experience New Zealand (Aotearoa). The platform enables visitors and travellers to browse regional destinations, explore granular location details, and submit authenticated ratings and reviews. Registered users can curate sequential, date-ordered travel itineraries across multiple destinations.

Architecturally, the application enforces a strict separation of concerns through a decoupled three-tier model. A standalone Angular single-page application functions purely as the presentation tier, consuming stateless JSON endpoints exposed by an Express 5 REST API. All communication between the frontend client and backend services is secured over TLS/HTTPS, ensuring data transit integrity while allowing the API tier to be tested, scaled, and managed independently of the client interface.

1.2. Technology Stack

- **Client Tier:** Built with Angular 22 as a Single Page Application (SPA) [1], leveraging standalone components, reactive signals for state management, Angular Router, and Vitest for unit testing.
- **Application Tier:** Implemented using Express 5 and TypeScript running directly on Node.js 26 via tsx, managed in production with PM2 process persistence under systemd.
- **Data Tier:** Managed PostgreSQL 16 on AWS RDS (ap-southeast-2) [2], accessed via Prisma 7 ORM utilizing the @prisma/adapter-pg driver over verified TLS connections.
- **Infrastructure:** Compute is hosted on Amazon Elastic Compute Cloud (Amazon EC2) [3], static assets are offloaded to Amazon Simple Storage Service (Amazon S3) [4], and relational state is delegated to Amazon Relational Database Service (Amazon RDS) for PostgreSQL [2].
- **Network:** Reverse proxy and TLS termination handled by Nginx with Let's Encrypt certificates [5], and DNS mapped to the instance's Elastic IP via DuckDNS (ourtearoa.duckdns.org). DuckDNS is a free static record, so no dynamic update client is needed [6].
- **Delivery and local development:** GitHub Actions tests and builds the client and deploys over rsync and SSH (see Section 6.4); Docker Compose runs PostgreSQL 16 for local development.

2. Background and Requirements Analysis

Visitors planning a trip around New Zealand gather destinations, opinions and dates from many separate sources. OurTearoa puts them in one place: browse a catalogue of places, read and write reviews, keep a shortlist and turn it into a dated itinerary. Browsing is open to everyone; anything personal needs an account. For the course, the same product has to show a working three-tier cloud deployment that markers can reach and test publicly.

2.1. Functional Requirements

ID	Requirement	Stories
FR-1	Public catalogue with region filter, name search and place detail (photo, map)	US-01 to US-03
FR-2	Public reviews; signed-in users create, edit and delete their own	US-04, US-09, US-10
FR-3	Account registration, login and account deletion	US-05, US-06
FR-4	Personal favourites and a dated itinerary	US-07, US-08, US-11 to US-13
FR-5	Place images stored in cloud object storage	US-03, US-14
FR-6	A documented REST API and health endpoint usable without the client	US-17, US-18

The full user stories US-01 to US-20 with their acceptance criteria and status are listed in Appendix B.

2.2. Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-1	Availability	Reachable at a public URL from submission until marking ends
NFR-2	Security	HTTPS everywhere, hashed passwords, no secrets in Git, least-privilege cloud access
NFR-3	Usability	Never a blank or frozen screen
NFR-4	Maintainability	Every merge to main deployed automatically and repeatably
NFR-5	Cost	Stay within the student free tier or credits
NFR-6	Testability	API testable independently of the UI

US-17 to US-20 in Appendix B cover evaluator access to the documented REST API, the health check, loading and error states, and HTTPS.

2.3. Constraints

- **Assignment brief (Option B):** three tiers; client and API decoupled by REST; at least 10 REST services including at least 3 POST; a cloud database, not a database installed on the VM; assets in cloud object storage; public URLs for client and API; Git collaboration without history rewriting. OurTearoa exposes 25 endpoints, 8 of them POST (Section 9.1), uses RDS and S3, and keeps a linear history of merged PRs.
- **Team:** two part-time students over one semester. One AWS account, administered by Mathias Jørgensen.
- **Budget:** free tier or student credits only, which rules out load balancers, Multi-AZ databases and managed container platforms.

- **Infrastructure:** the 1 GB t3.micro cannot build the Angular client and must not carry image traffic. Both shaped the design: the client is built in CI (Section 6.4) and images go straight to S3 (Section 5.5).

2.4. Assumptions

Assumption	Justification
Traffic is low: the team, markers and a few test users	Short outages outside marking sessions are acceptable
No SLA; availability matters during the evaluation window	Users are mainly in New Zealand and Australia
Product scope	Any signed-in user may edit places and add images
Community-curated catalogue; no moderation in scope	Stored personal data is minimal (name, email, password hash)
No payments or location tracking	Place images may be public

2.5. Cloud Opportunities

- **Managed database (RDS):** backups, patching and storage are AWS's job, not the team's.
- **Object storage with presigned uploads (S3):** durable image storage outside the VM; uploads bypass the API host completely [7].
- **Identity (IAM role):** the API gets short-lived S3 credentials from the instance role, with no access keys on disk [8].
- **Elastic CI capacity (GitHub Actions):** builds and tests run on hosted runners, so the production VM stays small [9].
- **Pay as you go:** the system runs on the smallest sizes now and has a clear path to scale (Section 7.4) when needed.

2.6. From Business Needs to Technical Objectives

Business need	Technical objective
Anyone can explore places	Public, read-only endpoints; auth only for writes
Travellers keep personal plans	JWT auth; ownership enforced on every user-owned row
Trustworthy and easy to reach	HTTPS on a named domain, HTTP redirected
Markers can assess without the UI	Every feature reachable through REST
Rich places without overloading the VM	Images in S3 via presigned direct upload
Always reflects the latest code	CI/CD on every merge, tests before deploy
Stays affordable	Free-tier-sized resources

3. Project Management

3.1. Work Breakdown Structure

The work was broken down into six work packages and tracked as GitHub issues . Each issue was implemented on a feature branch and merged through a reviewed pull request.

WP	Work package	Tasks (GitHub issues)
1	Project setup	Kick-off meeting and minutes; repository scaffolding and conventions; local hello-world slice
2	AWS infrastructure	Provision RDS PostgreSQL; backend database round-trip; provision and configure EC2; public API URL ; HTTPS for the client; S3 bucket and IAM role; secrets, RDS security and IAM access
3	CI/CD	GitHub Actions deploy; production migration fixes; client tests in CI ; Prisma client regeneration and serialised deploys
4	Backend REST API	JWT authentication; favourites, itinerary, reviews, users and places endpoints; S3 presigned image upload
5	Angular client	Core models, design tokens, auth store and interceptor; app shell, login, register, auth guard; catalogue, detail panel, filter, map, reviews; favourites, itinerary, profile; image upload
6	Testing, evidence and documentation	API curl script and public smoke test; evidence pack; design document; screenshots; report

Table 1: Work Breakdown Structure (WBS)

3.2. Schedule, Milestones & Sprint Progress

The schedule follows the issue history in GitHub (created and closed dates). The project ran from the kick-off on 6 August 2026 to the submission on 16 October 2026.

Period	Phase	Milestones reached
Start Aug 2026	Kick-off and planning	Start meeting and minutes; requirements and user stories; work packages 1 to 6 created as issues
Aug to Sep 2026	Cloud skeleton	RDS round-trip and public API URL live; client served over HTTPS and CI/CD deploying on every merge
Oct 2026	Backend API	JWT auth and all REST endpoints implemented; curl script for every endpoint; production migrations fixed
Oct 2026	Angular client and S3	All client screens; client tests in CI; S3 bucket and IAM role; image upload working end to end
Oct 2026	Evidence and design	Public smoke test log and evidence pack; requirements and quality sections; final AWS values and screenshots
Oct 2026	Report and video	Report written; submission on 16 October 2026

Table 2: Project timeline and milestone delivery schedule

Most of the feature work happened in the last two weeks, once the cloud skeleton was stable.

3.3. Meeting Minutes & Collaboration Logs

The project started with one formal kick-off meeting. After that, the team coordinated asynchronously on GitHub instead of holding recurring meetings: tasks were assigned through issues, questions and decisions were recorded as issue comments, and every change was discussed in a pull request review. This kept decisions traceable and fitted the two members' different schedules. The issue and pull request history therefore serves as the collaboration log.

Kick-off meeting, 6 August 2026 (about 4 hours; agenda in issue #1, minutes in issue #2)

- Agenda: define the project scope, determine roles, choose and challenge the tech stack, start the GitHub repository.
- Decisions: a map-based travel web app; Angular with Leaflet and OpenStreetMap for the client; a TypeScript/Express API on a single EC2 instance; AWS RDS PostgreSQL and S3 for data and images; GitHub Actions for CI/CD.
- Actions: Tim Schiller created the first work packages as issues (#3 to #12: scaffolding, RDS, EC2, public API, HTTPS, CI/CD, evidence); Mathias Jørgensen added tasks for RDS security and IAM access and took over the RDS provisioning (#5).

Coordination log (from GitHub issues and pull requests)

Date	Where	Decision or outcome
29-30 Sep 2026	#8, #9, #11	Cloud skeleton accepted: public API URL live, client served over HTTPS with a Let's Encrypt certificate, deploy on every merge. Mathias Jørgensen documented the HTTPS acceptance checklist in #9.
4 Oct 2026	#5, #29 to #44	Decision to store plain latitude and longitude instead of PostGIS, because no screen needs spatial queries (comment by Tim Schiller in #5). Backend API split into one issue per resource; Tim Schiller implements the endpoints and the curl smoke test.
5-6 Oct 2026	#30, #47	JWT_SECRET set on the server before the auth deploy. Production migration failure reported and fixed by Mathias Jørgensen.
6 Oct 2026	#48 to #70	Client work split into 23 issues (shell, auth, catalogue, detail panel, map, reviews, favourites, itinerary, profile, CI tests), created by Mathias Jørgensen. Most screens were then implemented by Tim Schiller and reviewed by Mathias Jørgensen in pull requests.
9 Oct 2026	#72, #87	Decision to store images in S3 instead of an upload directory on EC2 (#72 closed, #87 opened). Responsibilities agreed: Mathias Jørgensen sets up the bucket and IAM role, Tim Schiller builds the presigned upload in the API and client (#71, #73).
11 Oct 2026	#104, #106	Failing image upload diagnosed and fixed by attaching the IAM instance profile and S3 CORS (Mathias Jørgensen). Deploy workflow made reliable by regenerating the Prisma client and serialising runs (Tim Schiller).
11-12 Oct 2026	#108 to #129	Report and evidence split into issues: requirements and quality sections, final AWS values, screenshots, report sections. Mathias Jørgensen confirmed the AWS console values and reviewed and merged the report pull requests.

4. Worklog and Collaboration Evidence

4.1. Project Management Evidence (GitHub Projects / Kanban)

Instead of Trello, the team used GitHub Issues with a GitHub Projects Kanban board, so that tasks, code and reviews live in one place (issue list). Tasks from the WBS became issues with a short description, usually a checklist of acceptance criteria, and an assignee; the board shows each issue's status from backlog to done. Each issue was implemented on its own feature branch and closed by a pull request that references it ("Closes #N"), so every finished task links to the code that delivered it. GitHub records when an issue was opened, updated and closed, which provides the timestamps for the schedule in the previous section. Over the project, 63 issues were created and 61 pull requests merged.

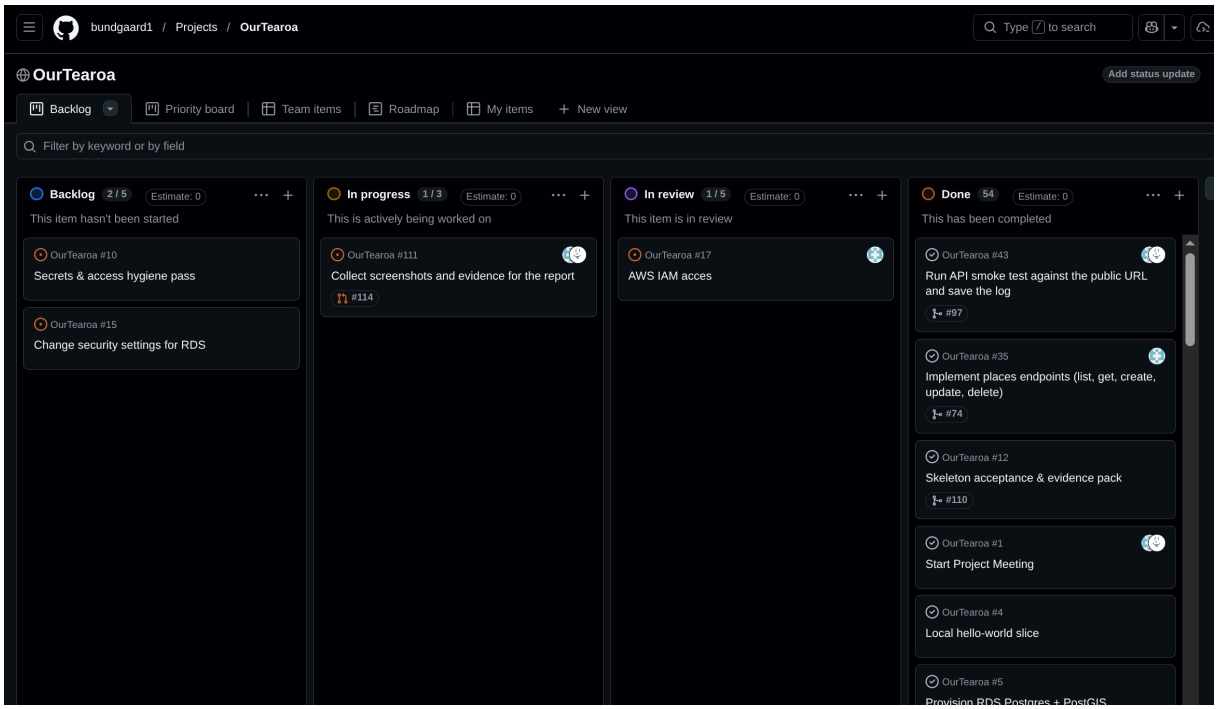


Figure 1: Kanban board of the GitHub project

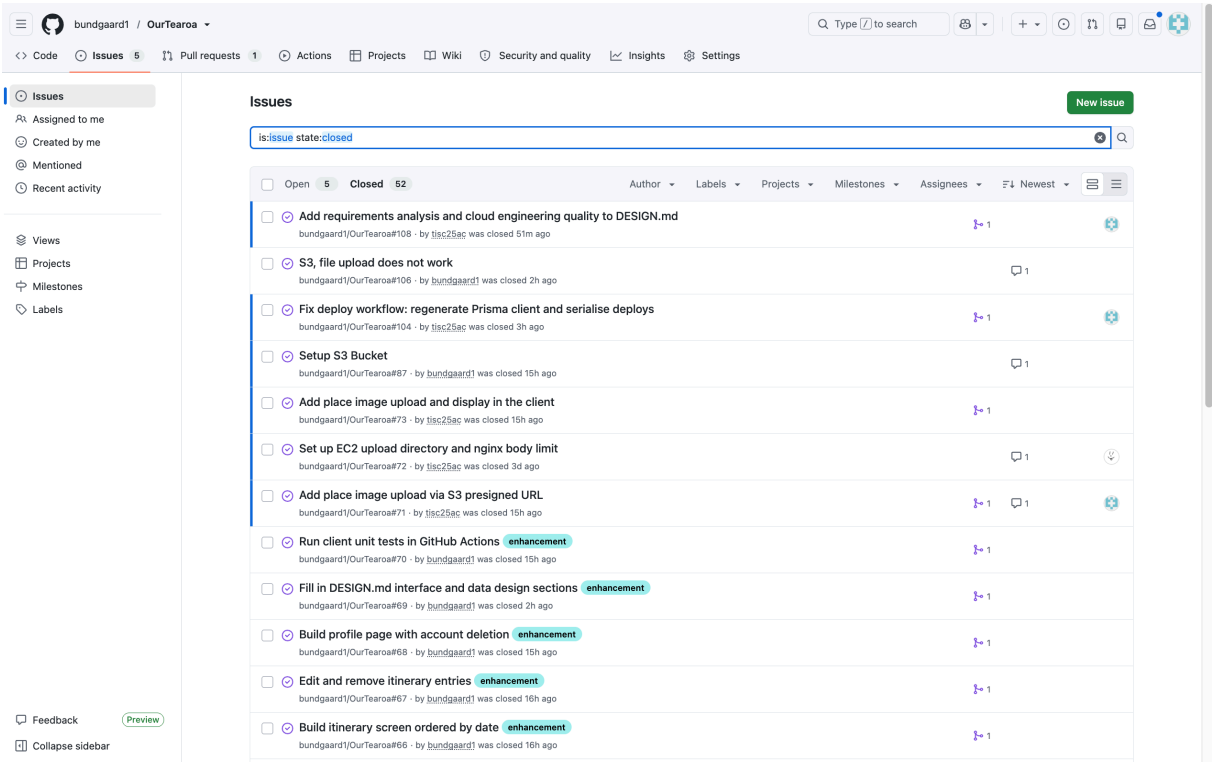


Figure 2: Closed GitHub issues; the pull request icon marks issues closed by a linked pull request

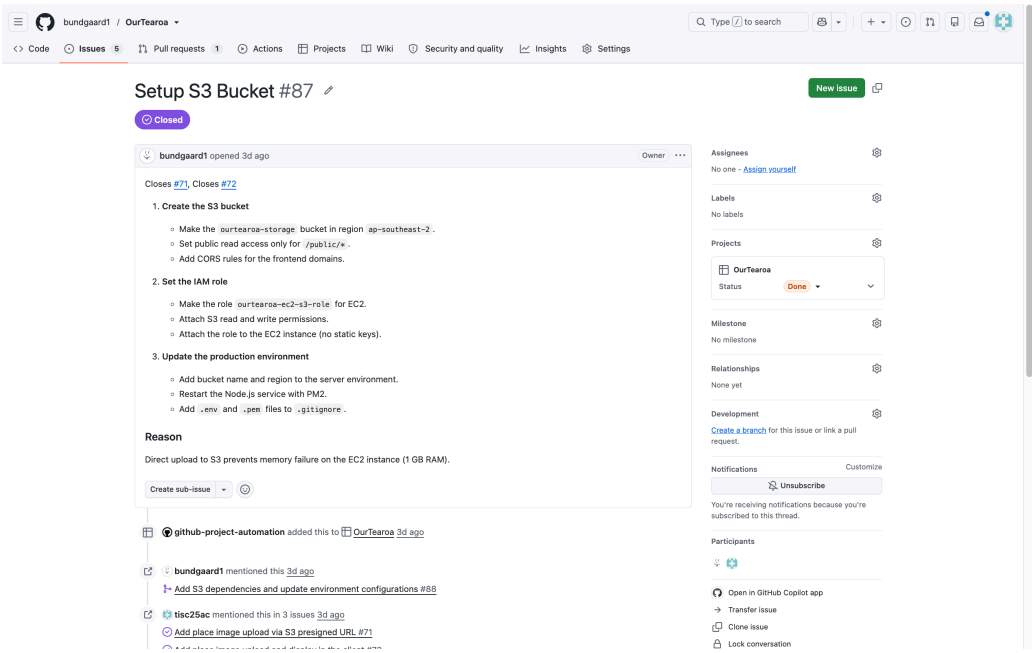


Figure 3: Example issue (#87, S3 bucket) with task checklist, project status and linked pull requests

4.2. Git Repository Contributions

Both members contributed throughout the project in the shared repository bundgaard1/OurTearoa. Excluding merge commits, main contains 46 commits by Mathias Jørgensen and 45 by Tim Schiller (git shortlog -sn –no-merges). The split by area reflects the division of work in the member contributions table:

Area (path)	Mathias Jørgensen	Tim Schiller
Client (client/)	9	20
Backend (backend/)	14	9
Cloud and CI/CD config (.github/, infra/)	7	2
Documentation and report (docs/, DESIGN.md)	23	17

Commits touching several areas are counted in each. Four further commits came from the GitHub Copilot coding agent (two CI fixes, see Appendix A). Work was done on feature branches and merged into main through 61 reviewed pull requests; main was never force-pushed and no commit history was rewritten, so the history shows each member's contributions as they happened.

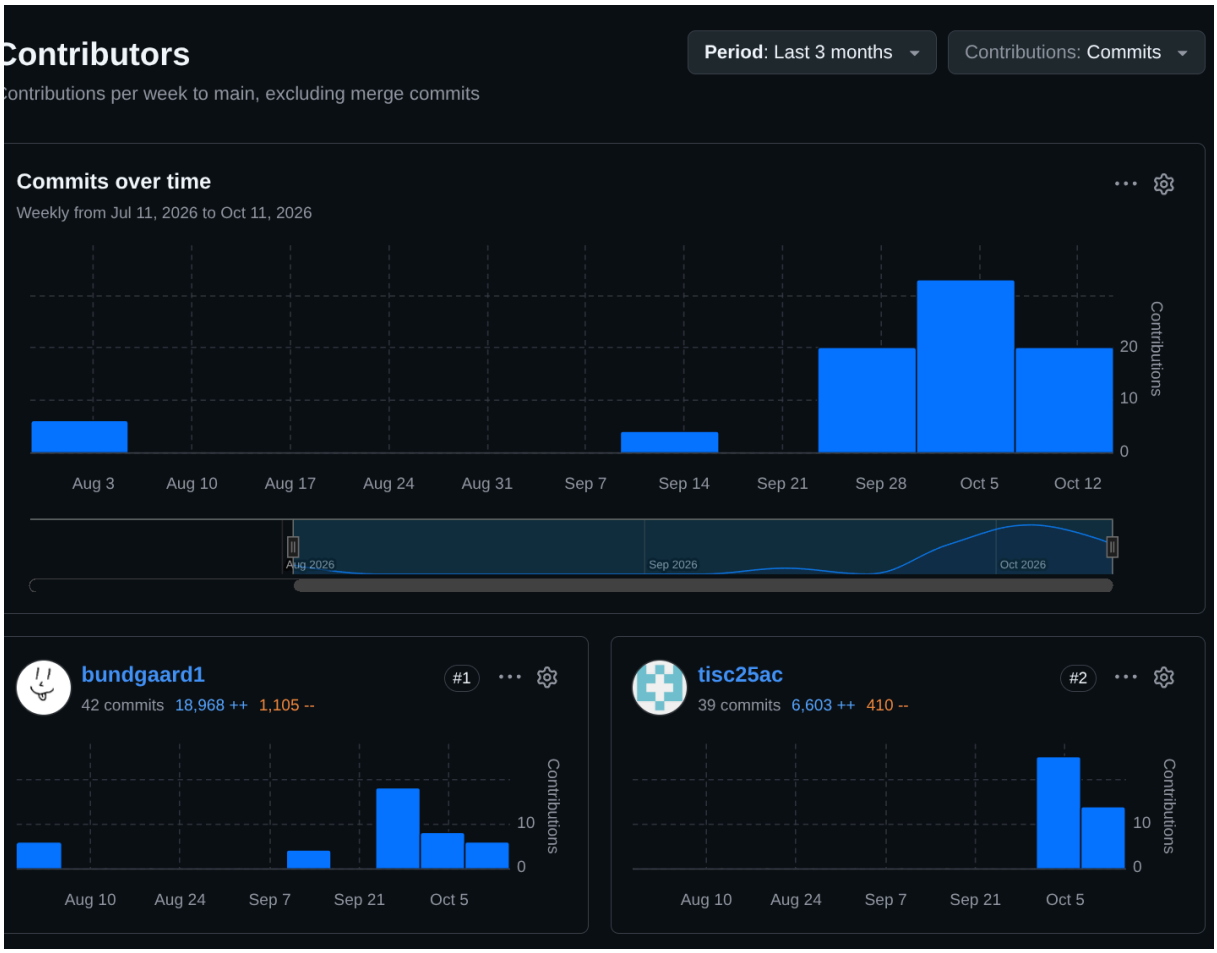


Figure 4: GitHub contributor graph

5. High-Level and Detailed Design

5.1. Three-Tier Architecture

The requirements this architecture answers are in Section 2; its security, availability, scalability and cost trade-offs are evaluated in Section 7.

Client tier - Angular 22 SPA

A standalone single-page application in client/, built on Angular standalone components with signals for state. It is a pure client-side app: there is no server-side rendering, so the compiled bundle in client/dist/client/browser/ is the entire deployable artefact. The API base URL is baked in at compile time from client/src/environments/ - there is no runtime configuration and no dev-server proxy, so changing the API host means recompiling and redeploying the client.

Application tier - Express 5 + TypeScript on Node

A REST API in backend/, written in TypeScript and run directly by tsx with no build step, so the deployed source and the running code are identical. Express 5 mounts every route under /api and delegates to controllers; cors() is enabled so the client can be served from a different origin during development. Prisma Client is the only data-access layer - no raw SQL outside the health check.

The application tier is stateless: all session state lives in the database, which is what allows the process to be reloaded or replaced without losing data.

Data tier - managed PostgreSQL

AWS RDS for PostgreSQL holds all application data. Because the database is a managed service rather than something installed on the VM, backups, patching and failover are handled by AWS, and the application tier holds no data of its own. The connection is TLS-verified against the AWS global CA bundle.

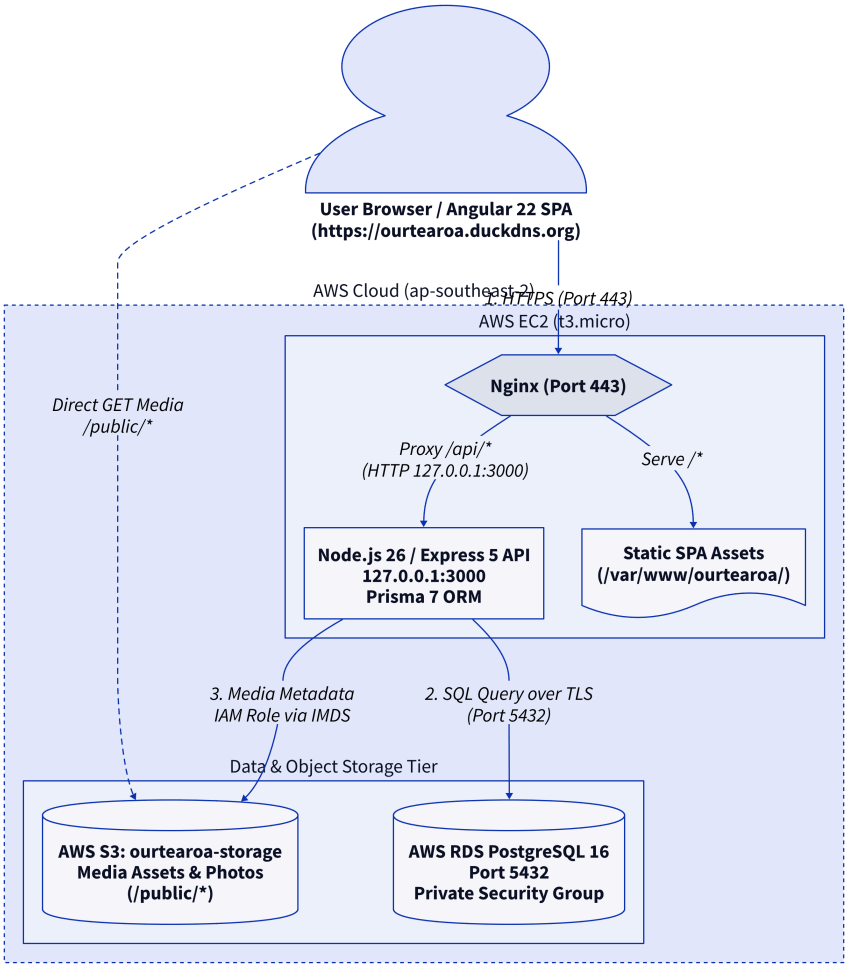


Figure 5: High-Level Three-Tier Cloud Architecture Diagram (EC2, Nginx, RDS, S3)

5.2. Request Lifecycle

A single place-detail page load, as an example:

1. The browser loads the SPA from Nginx over HTTPS.
2. The component queries its data, which has not been populated, and issues `GET /api/places/:id`.
3. Nginx terminates TLS and proxies `/api/*` to `127.0.0.1:3000`.
4. Express routes the request to the places controller.
5. The controller calls Prisma, which opens a TLS connection to RDS and runs the query.
6. The row is serialised to JSON and returned with `200`.
7. Any error is translated to a status code and JSON error body; a failed request surfaces a visible error state in the component rather than a blank screen.

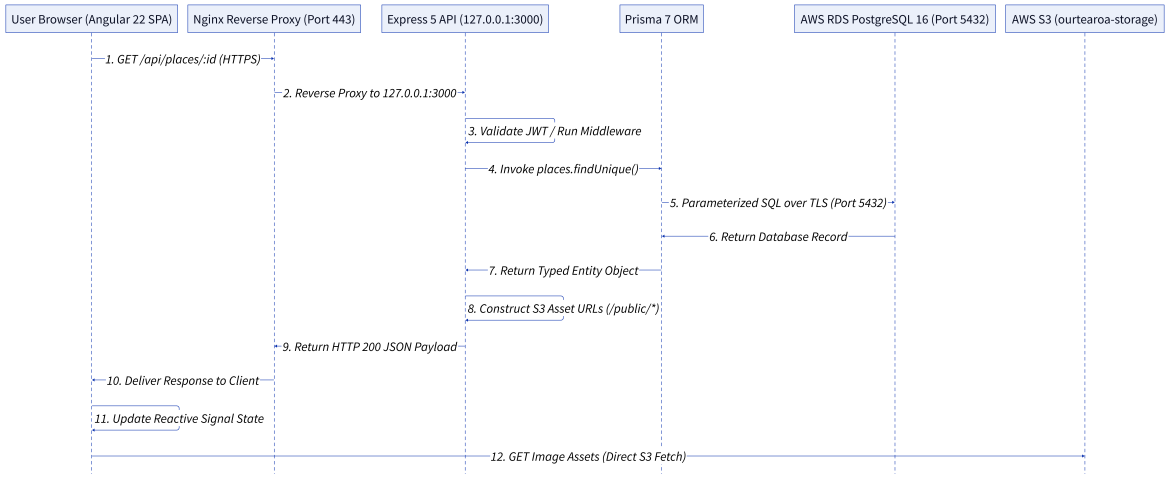


Figure 6: Request lifecycle of a place-detail page load

5.3. Interface Design

The frontend is deliberately minimal: plain standalone components, a single global stylesheet with CSS variables and no UI component library. The focus of the project is the cloud architecture, so the client only needs to exercise every API feature with clear loading, empty and error states (US-19).

#	Screen	Purpose	Route	Access
1	Place list / catalogue	Browse, search by name and filter by region	/places (/ redirects here)	public
2	Place detail	Photo, description, map, favourite toggle, “Plan a visit” form, photo upload and reviews	/places/:id (child of /places)	public; actions need login
3	Login	Authenticate an existing user	/login?returnUrl=...	public
4	Register	Create a new account	/register	public
5	Favourites	The user’s saved shortlist, with remove	/favorites	auth guard
6	Itinerary	Dated trip plan sorted by start date, with edit and remove	/itinerary	auth guard
7	Profile	Name, email, member since, account deletion	/profile	auth guard
8	Not found	Fallback for unknown URLs	**	public

Master/detail. /places/:id is a child route of /places, so the catalogue stays rendered and the detail panel opens beside it at half the window width. On screens of 768 px or less the panel covers the full screen. Closing the panel navigates back to /places. Because the panel is a route, a place link can be shared or bookmarked.

Wireframes. The screens were sketched as low-fidelity wireframes in the planning phase, each mapped to its user stories and the endpoints it calls.



Figure 7: Wireframes from the planning phase: place list with map, place detail, add place

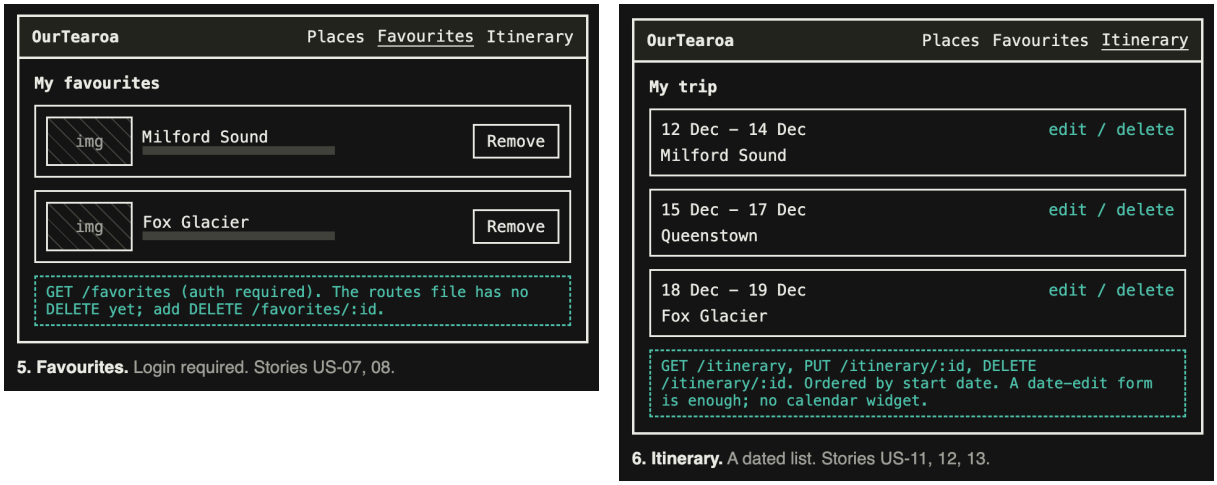


Figure 8: Wireframes from the planning phase: favourites and itinerary

The implementation kept the screens, the navigation and the endpoint mapping, but changed three things. The map moved from the place list into the detail panel, and the detail opens as a side panel next to the list instead of a separate page. The add-place screen was not built. Image upload moved to the detail panel, and the planned presigned PUT through a separate /uploads endpoint became a presigned POST, so that S3 enforces the size limit (Section 5.5).

5.3.1. Navigation and User Flows

The header is always visible. Visitors see **Places**, **Login** and **Register**; signed-in users see **Places**, **Favourites**, **Itinerary**, their name (links to **Profile**) and **Logout**.

Primary flow: browse, open a place, log in, save it as a favourite, plan a visit. The flow runs catalogue → detail panel → login with returnUrl → favourites / itinerary, and both the favourites list and the itinerary link back to the detail panel.

1. A visitor opens /places, searches by name or filters by region, and selects a card. The detail panel opens at /places/:id.

2. Reading is public. The favourite toggle, “Plan a visit”, the review form and the photo upload are shown only to signed-in users. In place of “Plan a visit” and the review form, visitors see a “Log in” link that carries returnUrl=/places/:id.
3. After login the user returns to that place via returnUrl. Registration always continues to /places.
4. “Save” adds the place to favourites in place, without leaving the panel. The same toggle is on every catalogue card.
5. “Plan a visit” creates an itinerary entry with start date, end date and an optional note. /itinerary lists the entries by start date.

Auth guard and session handling.

- Login stores the JWT (valid 7 days) and the user in localStorage; an HTTP interceptor adds Authorization: Bearer <token> to requests to the API base URL only, so the token is never sent to S3 or other hosts.
- /favorites, /itinerary and /profile use authGuard. A visitor opening one of them is redirected to /login?returnUrl=<requested URL> and lands there after logging in.
- If the API answers 401 (expired or invalid token) on a protected call, the interceptor clears the session and redirects to /login with the current URL as returnUrl.
- Logout and account deletion clear the session and return to /places.

5.4. Database Model & Relational Schema

The schema lives in backend/prisma/schema.prisma and is applied with Prisma migrations (backend/prisma/migrations/): migrate dev locally, migrate deploy in the deployment workflow. All primary keys are UUID strings generated by Prisma.

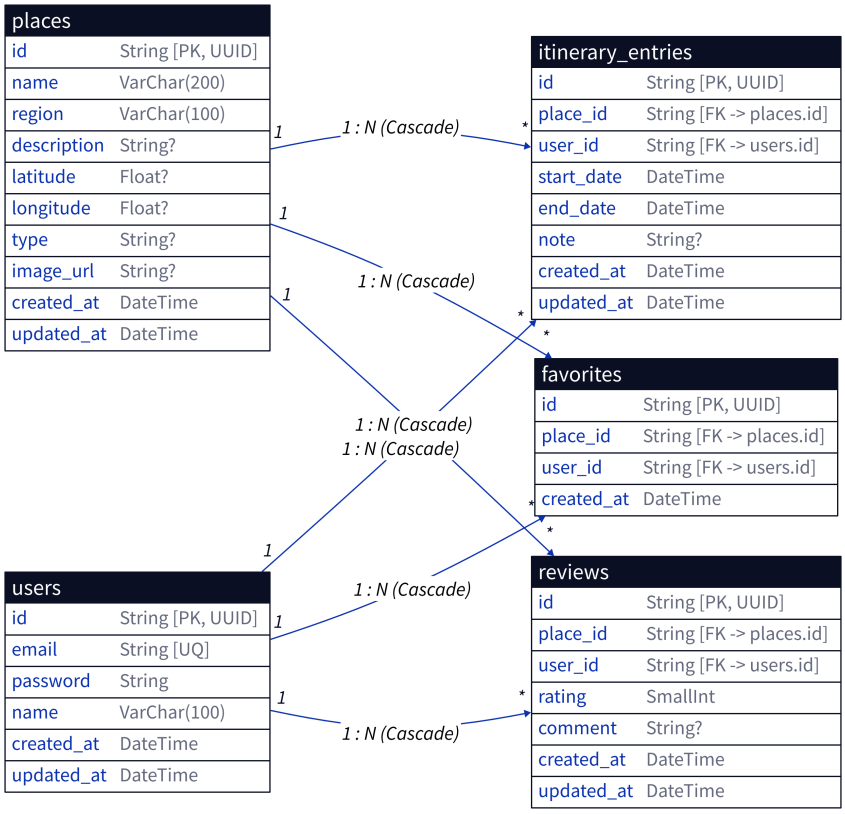


Figure 9: Entity-Relationship Diagram (ERD) of PostgreSQL Database Schema

5.4.1. Design Notes

- **Indexes.** Every foreign key is indexed, and Place.name and Place.region are indexed for search and filter.
- **Cascades.** Deleting a user (DELETE /users/me) removes their reviews, favourites and itinerary entries; deleting a place removes everything that references it. No orphan rows are possible.
- **Ownership.** Every user-owned row carries userId, and the API filters by the token's user. Another user's row answers 404, not 403, so IDs cannot be probed.
- **Images outside the database.** Only the URL is stored; the bytes are in S3 (Section 5.5). Replaced images are not yet deleted from the bucket.
- **Seed data.** backend/prisma/seed.ts loads the demo places, a demo user (with a hashed password) and sample reviews, favourites and an itinerary entry.

5.5. Object Storage Architecture (AWS S3)

Image uploads (S3). Image bytes never pass through the 1 GB EC2 instance. The client uploads straight to S3 with a presigned POST, and the API only signs and records:

1. POST /api/places/:id/image-upload-url with {contentType} (JPEG, PNG or WebP). The API generates the object key itself, public/places/<placeId>/<uuid>.<ext>, so a client filename is never used, and returns url, fields, key and the final imageUrl.

2. The browser sends a multipart POST of fields plus the file to url, with no Authorization header. The signed policy only allows that exact key, that content type and 1 byte to 5 MB (content-length-range). It expires after 5 minutes.
3. POST /api/places/:id/image with { key }. The API checks that the key belongs to that place and that the object exists (HeadObject), then saves Place.imageUrl = S3_PUBLIC_BASE_URL + "/" + key.
4. GET /api/places and /api/places/:id return imageUrl, and the browser loads the image directly from S3.

Configuration: S3_BUCKET_NAME, S3_PUBLIC_BASE_URL and AWS_REGION. If the bucket settings are missing, both endpoints return 503. If S3 itself fails, they return 502. Credentials come from the default AWS provider chain: the instance role on EC2, a profile or env credentials locally.

Trade-offs: a presigned POST was chosen over a presigned PUT because S3 enforces the size limit, not just the client. Public read on public/* keeps image URLs simple and cacheable but means every uploaded image is world-readable. Replaced images are not deleted from S3 yet. Like PUT /places/:id, any signed-in user may set a place image. HeadObject on a missing key returns 403 instead of 404 unless the role has s3:ListBucket, so the confirm step then answers 502 instead of 400.

6. Cloud Implementation / Setup Evidence

6.1. Compute Provisioning & System Hardening (AWS EC2)

Service	Purpose	Notes
EC2 t3.micro	Hosts Nginx and the Node API	1 GB RAM, so a swap file is configured to keep the Out-Of-Memory (OOM) killer away from the API process
DuckDNS	Public DNS for ourtearoa.duckdns.org	Free; the record points to the Elastic IP 54.252.56.209, which stays the same across stop and start, so no dynamic update client is needed. Route 53 was not used because it is a paid service

The EC2 instance serves 443, and 80 only to redirect to HTTPS. SSH on 22 is used for administration and by the deploy workflow. Nginx terminates TLS with a Let's Encrypt certificate (Certbot) [5], serves the SPA from /var/www/ourtearoa, and proxies /api/* to 127.0.0.1:3000. PM2 runs the API with systemd persistence.

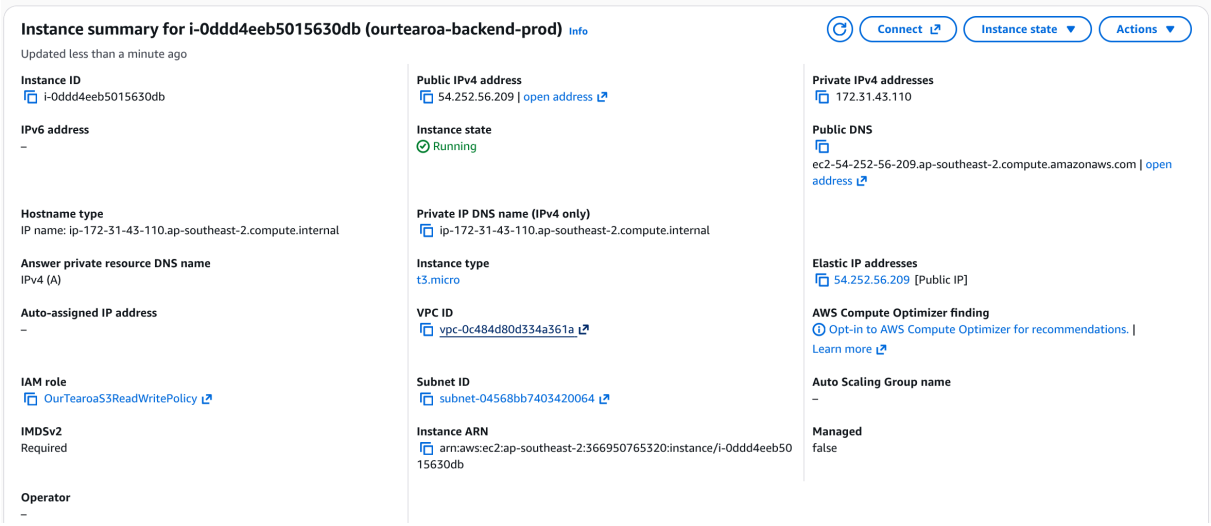


Figure 10: EC2 instance in the AWS console, taken during the initial setup (before the Elastic IP and the IAM role were attached)

```
[ec2-user@ip-172-31-43-110 ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           913Mi       641Mi        67Mi       2.0Mi       203Mi       132Mi
Swap:          2.0Gi       514Mi       1.5Gi
```

Figure 11: Swap file on the EC2 instance (`free -h`)

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/ourtearoa.duckdns.org/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/ourtearoa.duckdns.org/privkey.pem
This certificate expires on 2026-12-29.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for ourtearoa.duckdns.org to /etc/nginx/conf.d/ourtearoa.conf
Congratulations! You have successfully enabled HTTPS on https://ourtearoa.duckdns.org
```

Figure 12: Let's Encrypt certificate issued and deployed to Nginx by Certbot



Figure 13: After an instance reboot: Nginx active, the API back online under PM2, and the public health check returning 200

6.2. Managed Cloud Database Setup (AWS RDS PostgreSQL)

Service	Purpose	Notes
RDS PostgreSQL	All application data	db.t4g.micro in ap-southeast-2a (Sydney), PostgreSQL 16, TLS with global-bundle.pem; Single-AZ, storage encryption enabled, automated backups with 1-day retention; 20 GiB allocated storage

RDS carries its own inbound rule for 5432 for the EC2 security group. For development, docs/aws-rds-setup.md also allows individual developer /32 addresses, which needs the instance to be publicly accessible. This access is still in place at submission; removing it is the first future improvement (see Section 7.1). Postgres credentials and the DATABASE_URL are supplied by the process environment, never committed.

The RDS connection uses sslmode=verify-full with global-bundle.pem, so the certificate is validated rather than merely encrypted.

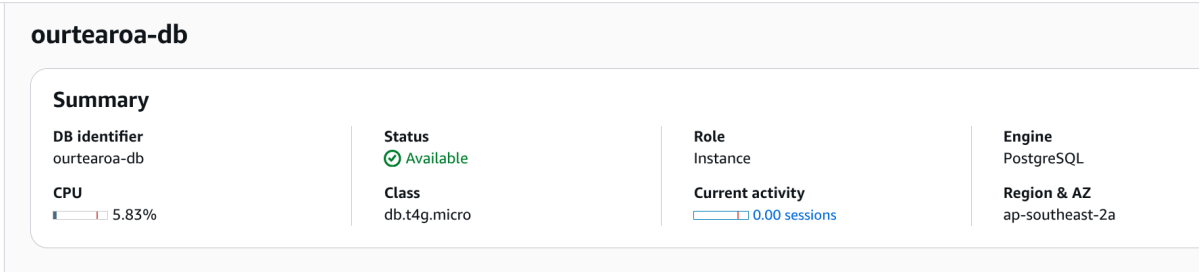


Figure 14: RDS instance ourtearoa-db in the AWS console

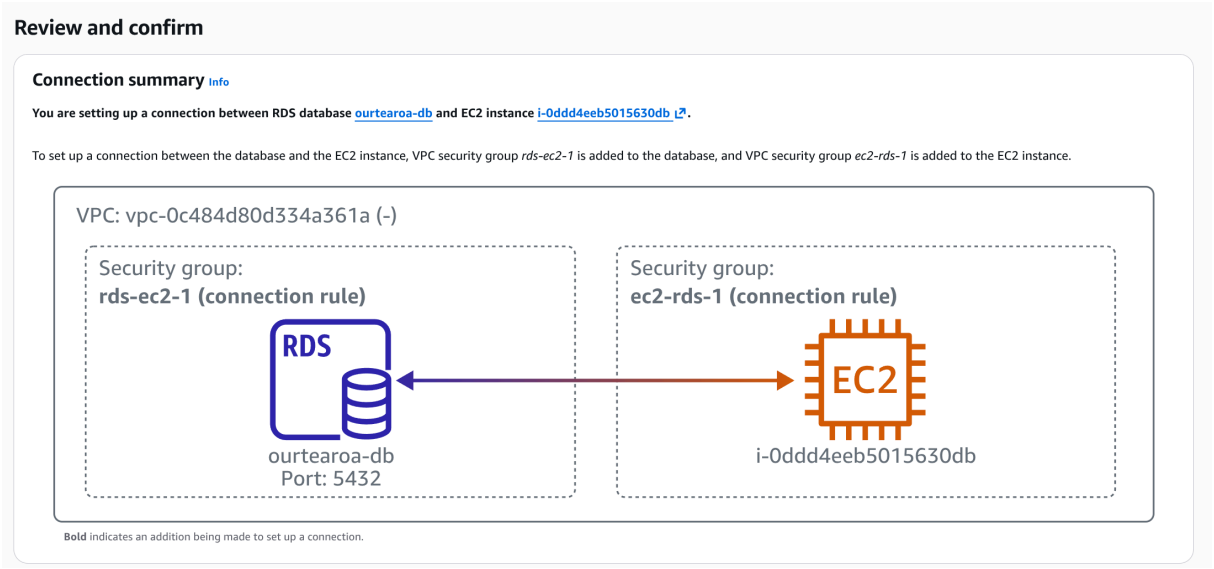


Figure 15: Security groups connecting the EC2 instance and RDS

6.3. Cloud Object Storage Provisioning (AWS S3)

Service	Purpose	Notes
S3 bucket	Place images	ourtearoa-storage in ap-southeast-2; public read only on public/*; CORS for the frontend origin; EC2 accesses it through the IAM role ourtearoa-ec2-s3-role (no static keys), set up in issue #87; end-to-end upload working since issue #106, and live imageUrl values point to this bucket

The upload flow is described in Section 5.5.

6.4. CI/CD Pipeline & Automated Deployment

The workflow runs on pushes to main that change files under backend/ or client/ and triggers .github/workflows/deploy.yaml, which:

1. Compiles the Angular production bundle on the GitHub runner. This is done in CI deliberately - the t3.micro does not have the memory to build the client.
2. Rsyncs the compiled bundle to the Nginx web root at /var/www/ourtearoa/.
3. SSHes into the instance, git pulls the backend, installs production dependencies, regenerates the Prisma client (backend/generated is not committed), and runs prisma migrate deploy. Both use npx –no, so the locked Prisma version is used and is never silently downloaded.
4. Reloads the API through PM2 and restores the SELinux httpd_sys_content_t context on the web root so Nginx can read the new files.

A concurrency group makes deploys run one at a time, because two overlapping runs on the same instance delete each other's node_modules during npm ci.

Migrations run as migrate deploy - never migrate dev - so production schema changes are applied, not regenerated. Secrets (the SSH key, host, user) are held in GitHub Actions secrets and never appear in the repository.

Health verification after any deploy:

```
curl -i https://ourtearoa.duckdns.org/api/health
```

200 with {"status":"UP","database":"UP"} confirms the API and its database connection are live. A 503 with reason: "timeout" means the RDS security group does not cover the instance; reason: "unreachable" means credentials or SSL settings are wrong.

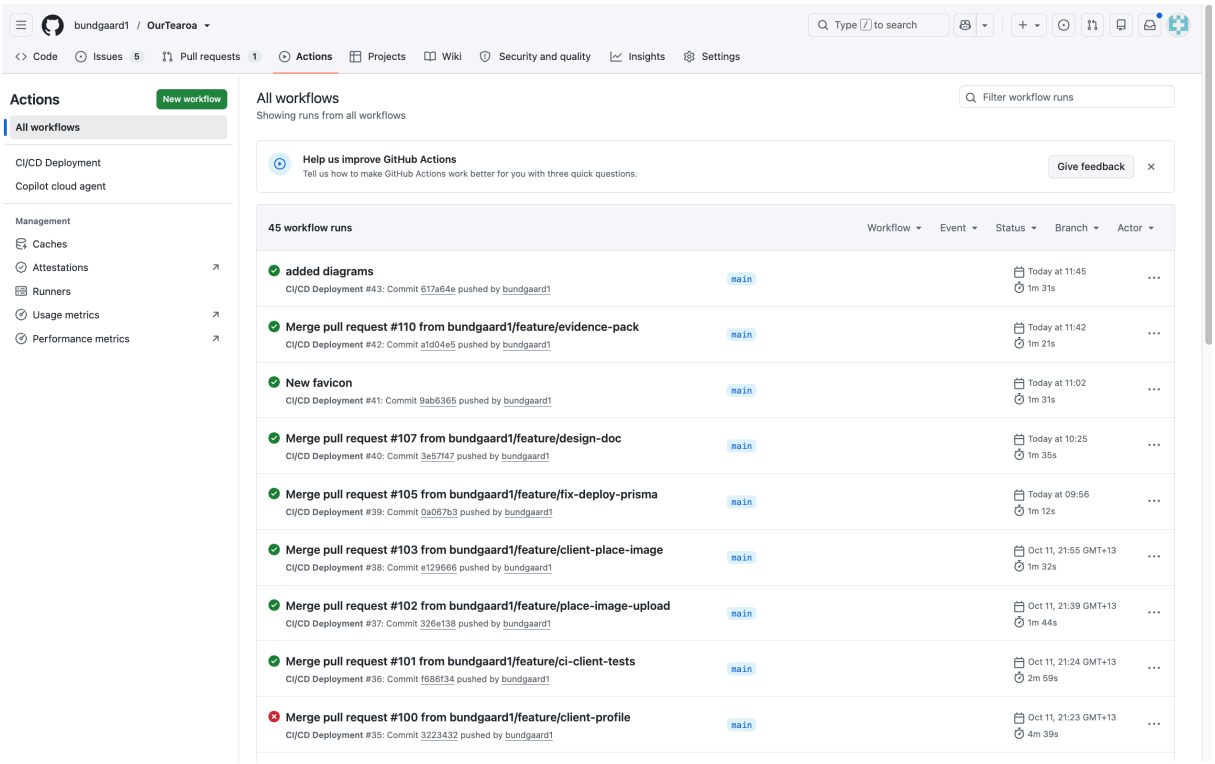


Figure 16: GitHub Actions: CI/CD deployment runs, one per merge to main

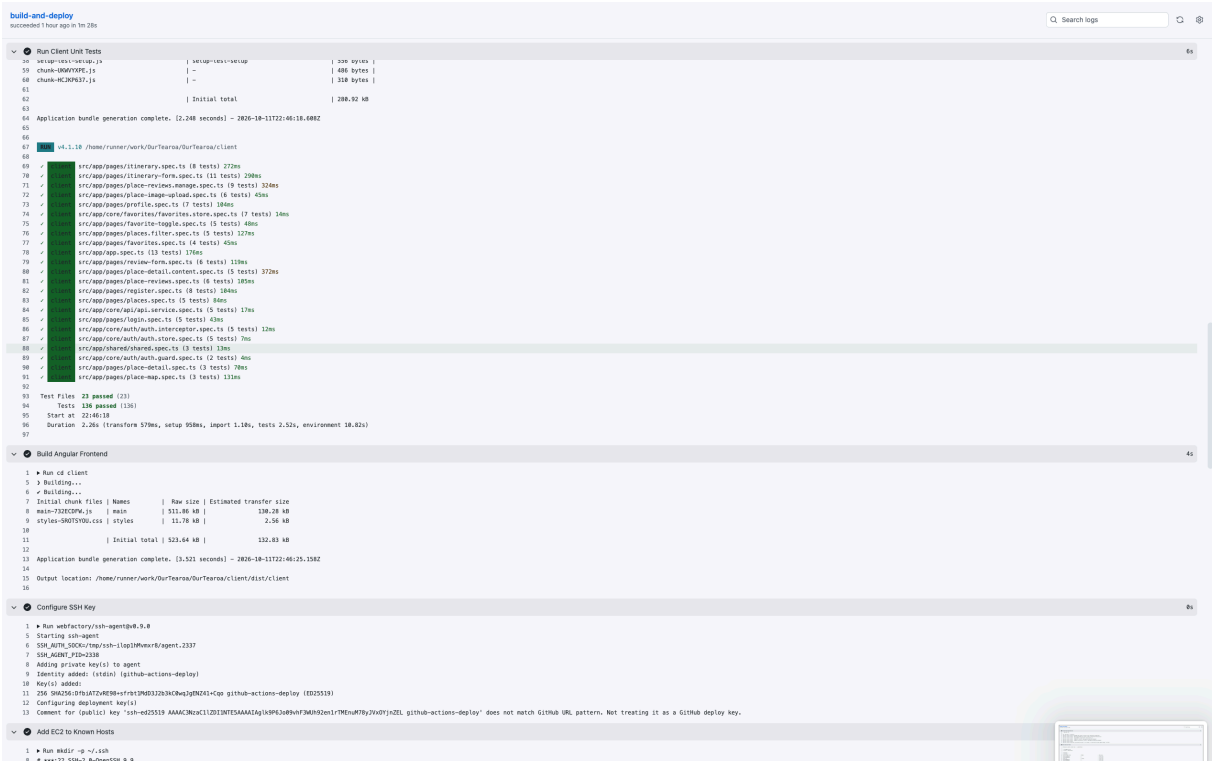


Figure 17: Deploy run: client unit tests (136 tests in 23 files passed) and production build before deployment

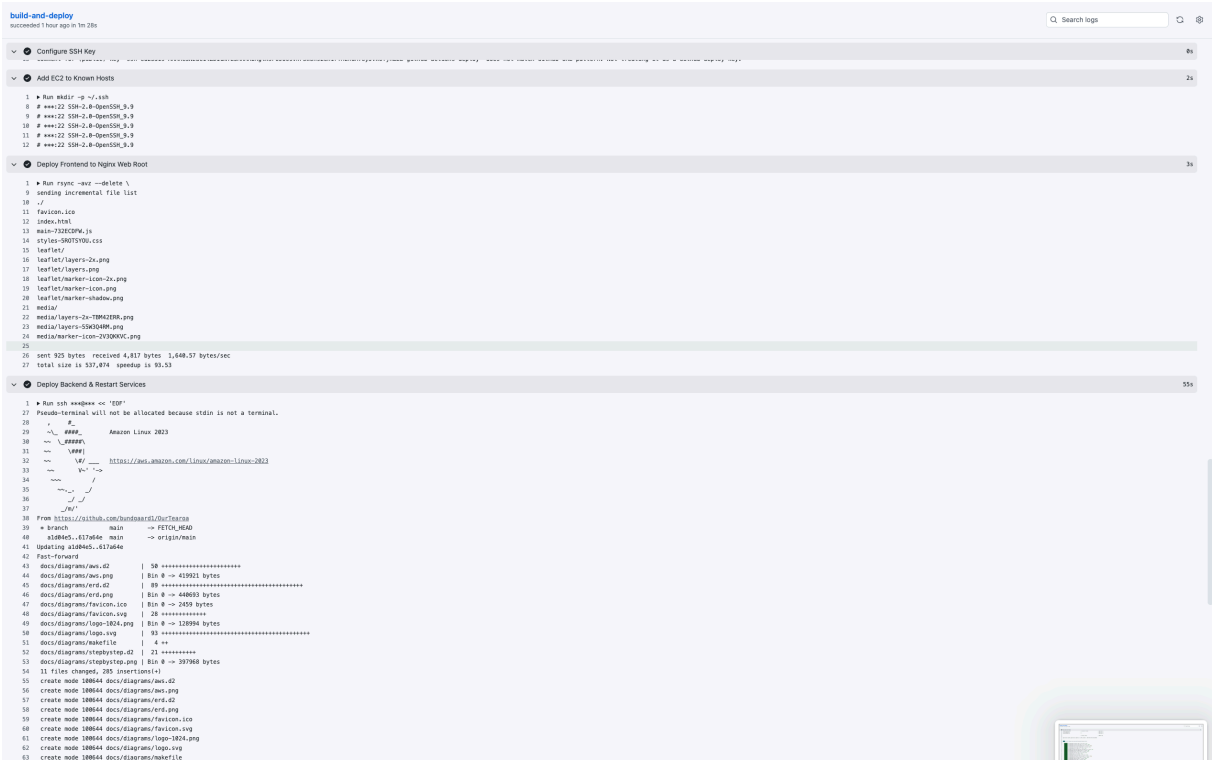


Figure 18: Deploy run: compiled client copied to the Nginx web root with rsync

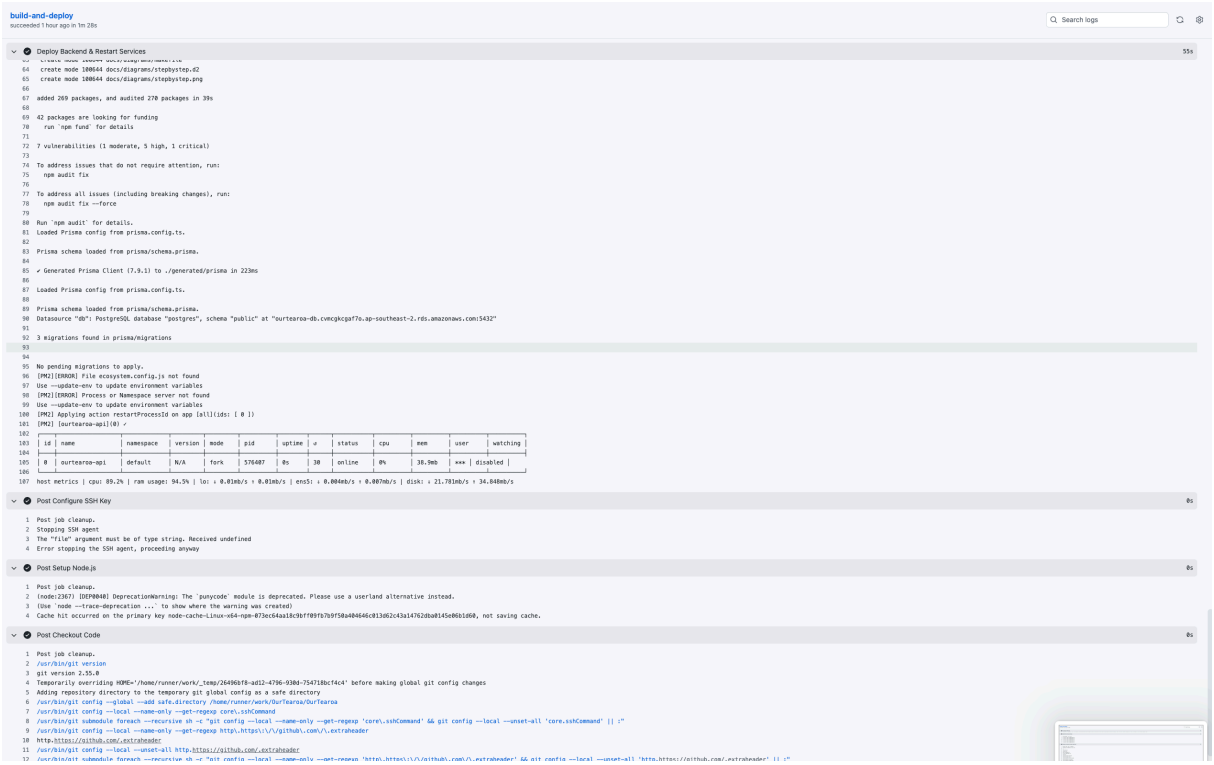


Figure 19: Deploy run: Prisma client generated, `prisma migrate deploy` ("No pending migrations to apply") and the API restarted under PM2

7. Cloud Engineering Quality

This section evaluates the deployed system against the quality attributes in Section 2 and states the trade-offs made. Console values were confirmed by Mathias Jørgensen in pull request #109.

7.1. Security

Layer	Current controls	Known gaps and next steps
Network	EC2 security group opens 443, 80 (redirect only) and 22; RDS allows 5432 from the EC2 security group	Developer /32 rules require RDS "Publicly accessible = Yes"; remove them and use an SSH tunnel or bastion. SSH could be limited to known addresses or replaced by SSM Session Manager
Data in transit	HTTPS with Let's Encrypt, HTTP redirected with 301; RDS over TLS with sslmode=verify-full; S3 over HTTPS	Add HSTS in Nginx
Data at rest	Passwords stored as bcrypt hashes (cost 10); images in S3	RDS storage encryption is enabled; S3 encrypts new objects by default (SSE-S3)
Identity and access	EC2 uses the IAM role ourtearoa-ec2-s3-role for S3, so no access keys exist on the server; presigned POST limits key, type, size and lifetime	Scope the role to the bucket and the needed actions only; give each team member their own IAM user with MFA instead of the shared administrator account
Application	JWT (7 days) for every write; ownership checked on every user-owned row (404 for others); input validated in controllers; Prisma parameterises all queries	CORS is open to any origin; no rate limiting on login or upload; no security headers (helmet); the token is in localStorage, which is exposed to XSS. Any signed-in user may edit places (US-15 partial)
Secrets	.env and keys are gitignored; deploy credentials are GitHub Actions secrets; only .env.example and the public RDS CA bundle are committed	A full audit of the repository history for leaked secrets was not done; AWS Secrets Manager or SSM Parameter Store would replace the .env on the instance
Operating system	Amazon Linux 2023 with SELinux enforcing; Nginx is the only public process, the API listens on 127.0.0.1:3000	Regular dnf upgrade is manual

The controls above cover the risks of a small public catalogue with minimal personal data. Rate limiting, a WAF and a private-subnet database were left out to stay within time and the free tier, and are listed in Section 7.7.

7.2. Availability and Reliability

Current design. A single EC2 instance in one Availability Zone serves the client and the API. Reliability comes from restarting rather than redundancy:

- PM2 restarts the API if it crashes, and its systemd unit starts PM2 again after a reboot. Nginx is a systemd service as well.
- The swap file protects the API from the OOM killer on 1 GB RAM.
- RDS is a managed service; AWS replaces failed hardware and applies patches in the maintenance window. The instance is a Single-AZ db.t4g.micro in ap-southeast-2a (Multi-AZ is off).

- GET /api/health checks the database with a timeout (HEALTH_DB_TIMEOUT_MS, default 3000 ms) and returns 503 with a reason, so a failure can be told apart from a network or credential problem (Section 6.4).
- Deploys are serialised and tests run first. PM2 restarts the process, so a deploy causes a downtime of a few seconds.

Expected availability (estimate). The instance, its AZ and the database are each single points of failure. AWS offers 99.5% for a single EC2 instance and 99.5% for a Single-AZ RDS instance; together that is roughly 99% per month, or up to about 7 hours of possible downtime. That is acceptable under the assumption in Section 2 that short outages outside marking sessions do not matter.

Path to high availability.

1. CloudWatch alarm or an external uptime check on /api/health, and EC2 auto-recovery on a failed status check. Low cost, first step.
2. An Application Load Balancer and an Auto Scaling Group across two AZs, with an AMI or launch template replacing the manual setup. This requires moving TLS from Certbot to ACM on the load balancer and the domain to Route 53.
3. RDS Multi-AZ for automatic database failover (typically one to two minutes).

7.3. Backup and Disaster Recovery

Component	Where the state lives	Backup	Recovery
Application data	RDS	Automated daily snapshots and point-in-time restore, retention 1 day; manual snapshot before risky migrations	Restore to a new instance and update DATABASE_URL
Schema	Git (backend/prisma/migrations/)	Every migration is versioned	prisma migrate deploy
Images	S3	99.99% designed durability [4], stored across several AZs	Versioning is disabled; enabling it would protect against accidental deletes and overwrites
Code and client	GitHub	Every merged PR	The deploy workflow rebuilds and redeploys
EC2 instance	None (stateless)	Setup documented in docs/cloud-ec2-setup.md	Launch a new instance, follow the guide, move the Elastic IP to it, so DNS and the deploy host secret stay unchanged

Targets:

- RPO: about 5 minutes for the database, the interval of RDS transaction log backups; zero for code.
- RTO: about 1 hour to rebuild the EC2 instance by hand and 30 to 60 minutes for an RDS point-in-time restore. Everything runs in ap-southeast-2, so a region-wide outage is not covered; cross-region snapshot copies and S3 replication would be the next step if that mattered. A restore has not been rehearsed yet.

With a retention of 1 day, point-in-time restore only reaches back 24 hours: a bad migration or deletion noticed later cannot be undone. Raising the retention to 7 days is the cheapest improvement here, since backup storage up to the database size is free.

Risk: backend/prisma/seed.ts deletes all rows before seeding and must never be run against RDS.

7.4. Scalability and Elasticity

What scales well already.

- The API is stateless: sessions are JWTs, not server memory, so more instances can be added behind a load balancer without sticky sessions.
- Image bytes go straight between the browser and S3, which scales on its own; the API only signs and records.
- The client is static files, ready to move to S3 and CloudFront [10].
- GitHub Actions provides build capacity on demand, so the production instance does not need to be sized for builds.

Current limits. One t3.micro (2 vCPU burstable, 1 GB RAM) with no automatic scaling. Burstable CPU credits can run out under sustained load. Scaling today means manual vertical scaling: stop the instance and change the instance type, with downtime. RDS can be resized the same way.

Scale-out path.

Load grows in	Response
Static client and images	CloudFront in front of S3 and the SPA; caches at the edge and removes static traffic from the EC2 instance
API requests	ALB and Auto Scaling Group with a target-tracking policy on CPU or request count
Database connections	Limit the Prisma pool per instance, or add RDS Proxy
Database reads	Larger instance class, then a read replica for the public catalogue

Trade-off. Elasticity was not built because the expected load (assumptions in Section 2) does not need it and an ALB alone would cost more than the rest of the system together (Section 7.6). The stateless design keeps the option open.

7.5. Performance

- **Region.** Everything runs in ap-southeast-2 (Sydney), close to the expected users in New Zealand and Australia, and the API and database are in the same region.
- **Queries.** Region filter and name search are done in the database (mode: insensitive), not in the client. Foreign keys used for lookups are indexed (placeId on reviews and favourites), and itineraries use the composite index (userId, startDate) that matches their sort order (Section 5.4).
- **Images.** Loaded directly from S3, not through Nginx or Node; the 5 MB limit keeps uploads bounded.

- **Client.** Production build from CI, served as static files by Nginx; the client loads data on demand per page.
- **Observability.** The health endpoint reports latencyMs for the database check.

Not yet done: no CDN or HTTP caching headers, no pagination on GET /places (fine for the current catalogue size), no load test. Name search with contains cannot use a normal index; a trigram index would be needed for a large catalogue.

7.6. Cost Optimisation

Monthly estimate (on-demand prices for ap-southeast-2, 730 hours, taken from public AWS price lists in October 2026; estimates, not billing data):

Item	Estimate (USD/month)	Note
EC2 t3.micro	about 9$	$0.0136 per hour
EBS root volume	about 2$	20 GB gp3 at about $0.10 per GB-month
RDS PostgreSQL	about 11$	db.t4g.micro Single-AZ at $0.016 per hour
RDS storage	about 3$	20 GiB at roughly $0.14 per GB-month (estimate); backup storage up to the database size is free
Elastic IP	about 3$	$0.005 per hour; AWS charges every public IPv4 address, including an attached Elastic IP
S3	under 0.10$	A few MB of images and low request volume
Data transfer	close to 0	Low traffic
DuckDNS, Let's Encrypt	0	Free services
GitHub Actions	0	Free for public repositories
Total	**about 30**	**Before free tier or credits**

The account is on the AWS free tier. th

Decisions taken for cost.

- Smallest instance sizes; the client build moved to CI so the instance can stay at 1 GB.
- No ALB, NAT gateway, Multi-AZ database or CloudFront; each would add a fixed monthly cost larger than the current need.
- Images in S3 rather than larger EBS volumes or database rows.
- Free DNS and certificates.

Recommended. An AWS Budgets alert at a small threshold (for example $10), deleting replaced images or adding an S3 lifecycle rule, and stopping or deleting all resources after marking ends.

7.7. Trade-off Summary and Future Improvements

Decision	Benefit	Cost or risk accepted
Single EC2 instance, no load balancer	Cheapest, simple to operate	Single point of failure, no automatic scaling (Section 7.2, Section 7.4)
Managed RDS instead of Postgres on the VM	Backups, patching, required by the brief	Higher cost than self-hosting

Decision	Benefit	Cost or risk accepted
Public RDS endpoint for development	Developers can run migrations and inspect data directly	Larger attack surface; not yet removed
Client built in CI	Production VM stays small	Deploy depends on GitHub Actions
Presigned POST direct to S3	No image traffic on the VM, size enforced by S3	Images are public; replaced files stay in the bucket
JWT in localStorage	Simple, stateless, works across tabs	Exposed to XSS; no server-side logout
Let's Encrypt and DuckDNS	Free HTTPS on a named domain	Depends on free external services
PM2 restart on deploy	Simple and reliable	A few seconds of downtime per deploy

Future improvements, in priority order.

1. Access hardening (not completed within the project): make RDS private and remove the developer /32 rules, give each team member a least-privilege IAM user with MFA, and audit the repository history for secrets.
2. Monitoring: CloudWatch alarms on the health endpoint, CPU and RDS storage, plus an AWS Budgets alert.
3. Application hardening: restricted CORS origin, helmet, rate limiting on login and upload, HSTS.
4. Run the tests on every pull request, not only before deploy.
5. RDS backup retention from 1 to 7 days, S3 versioning, deleting replaced images, and a rehearsed RDS restore.
6. CloudFront for the client and images.
7. ALB with an Auto Scaling Group across two AZs and RDS Multi-AZ, when traffic or availability needs justify the cost.

8. User Manual

8.1. Accessing the Application

Open https://ourtearoa.duckdns.org in a current desktop or mobile browser (for example Chrome, Firefox, Safari or Edge). No installation is needed. The site is served over HTTPS only; a request to http:// is redirected to https://. The address bar shows a valid Let's Encrypt certificate .

Browsing places and reading reviews works without an account. Saving favourites, planning visits, writing reviews and uploading photos require a free account (Step 2). On screens of 768 px or less the place detail covers the full screen.

If a page cannot load its data, it shows an error message with a **Retry** button instead of a blank screen. Whether the backend is up can be checked directly at https://ourtearoa.duckdns.org/api/health, which returns {"status":"UP","database":"UP"}.

8.2. End-User Workflow Walkthrough

Step 1: Browse and filter places. The start page /places lists every place as a card. Type in *Search* to filter by name, or choose a *Region* (default *All regions*). *Clear filters* resets both. Select a card to open the detail panel next to the list. It shows the photo, region, type, description, a map (OpenStreetMap) when coordinates are known, and the reviews. *Close* returns to the list. The panel has its own URL (/places/:id), so a place can be bookmarked or shared.

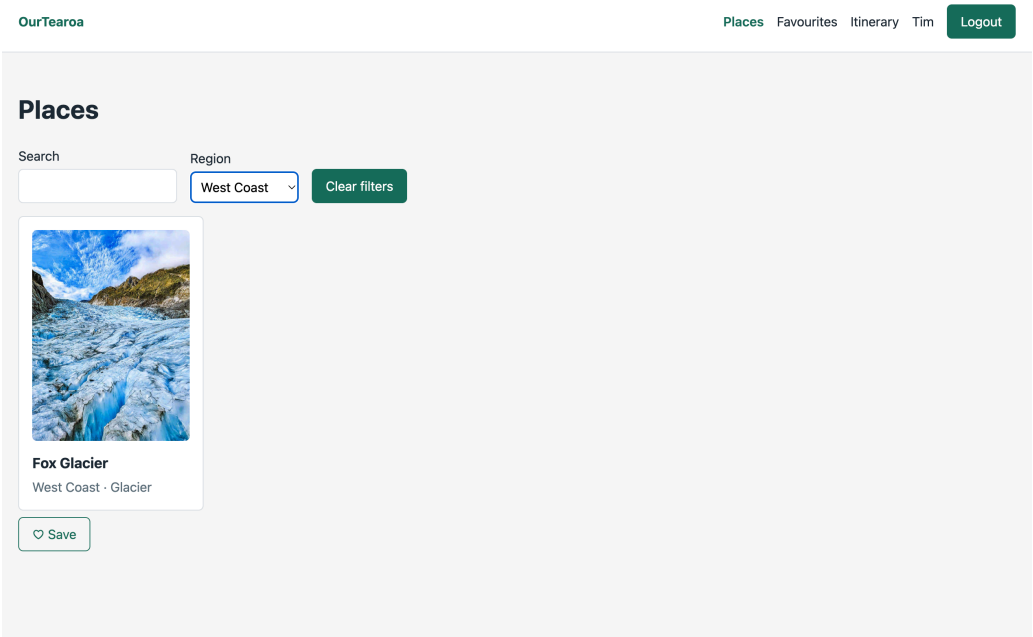


Figure 20: Place catalogue filtered by region (West Coast)

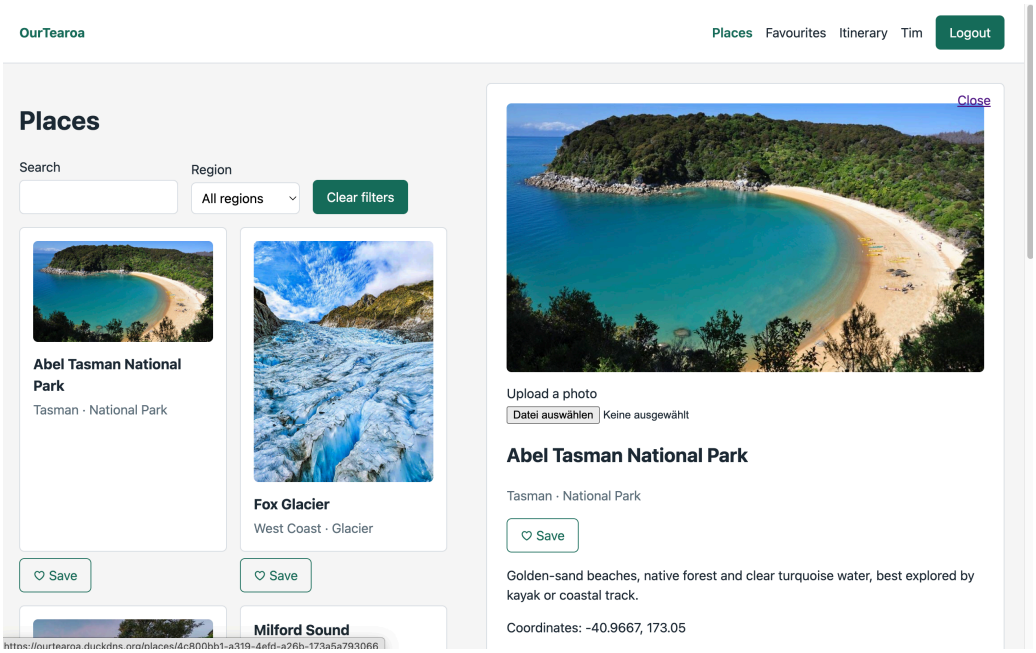


Figure 21: Place detail panel opened next to the catalogue

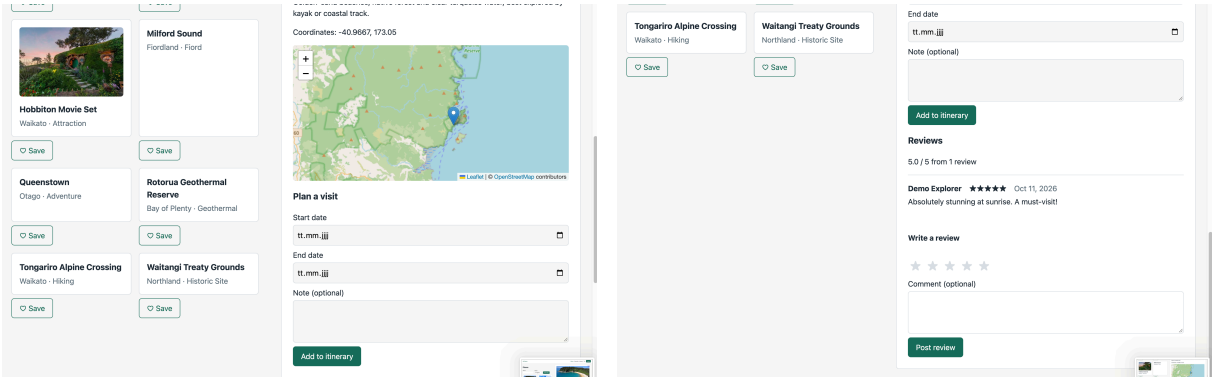


Figure 22: Detail panel continued: map and "Plan a visit" form (left), reviews and review form (right)

Step 2: Create an account and log in. Select *Register* in the header, enter name, email and a password of at least 8 characters, and select *Create account*. You are signed in straight away. Returning users select *Login*. If you choose *Log in* from a place's detail panel, you return to that place afterwards. A session lasts 7 days; after that, the next action asks you to log in again. *Logout* in the header ends the session.

Step 3: Save favourites. On any card or detail panel, select ♡ *Save*. The button changes to ♥ *Saved*; selecting it again removes the favourite. *Favourites* in the header lists all saved places, each with *Remove* and a link back to its detail panel.

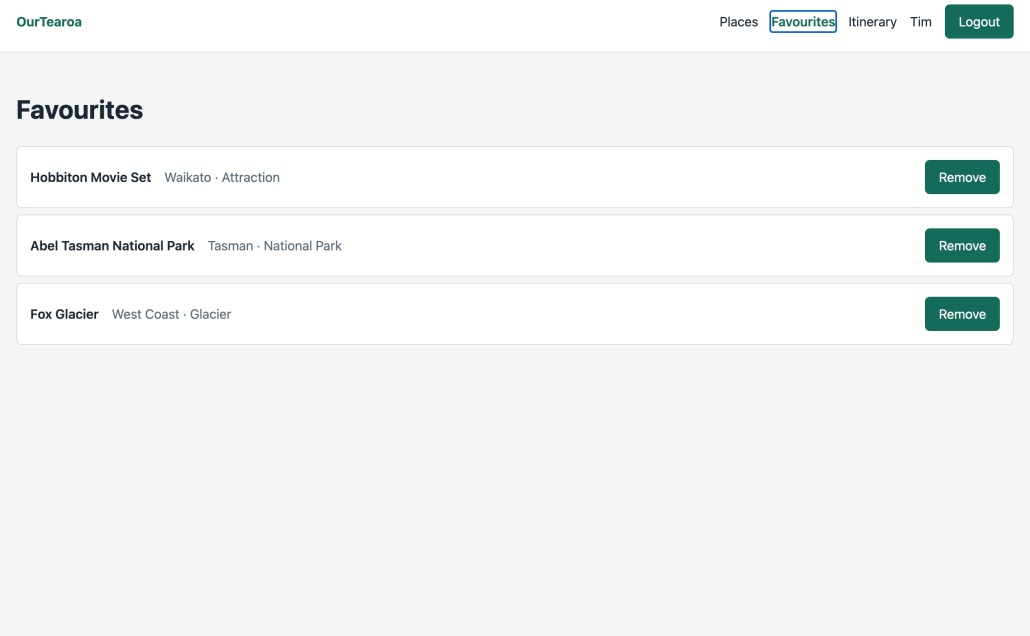


Figure 23: Favourites page

Step 4: Plan a visit. In the *Plan a visit* section of the detail panel, choose a start date and an end date (the end date must not be before the start date), add an optional note, and select *Add to itinerary*. *Itinerary* in the header lists all entries ordered by start date. Each entry can be changed with *Edit* (*Save changes*) or deleted with *Remove*.

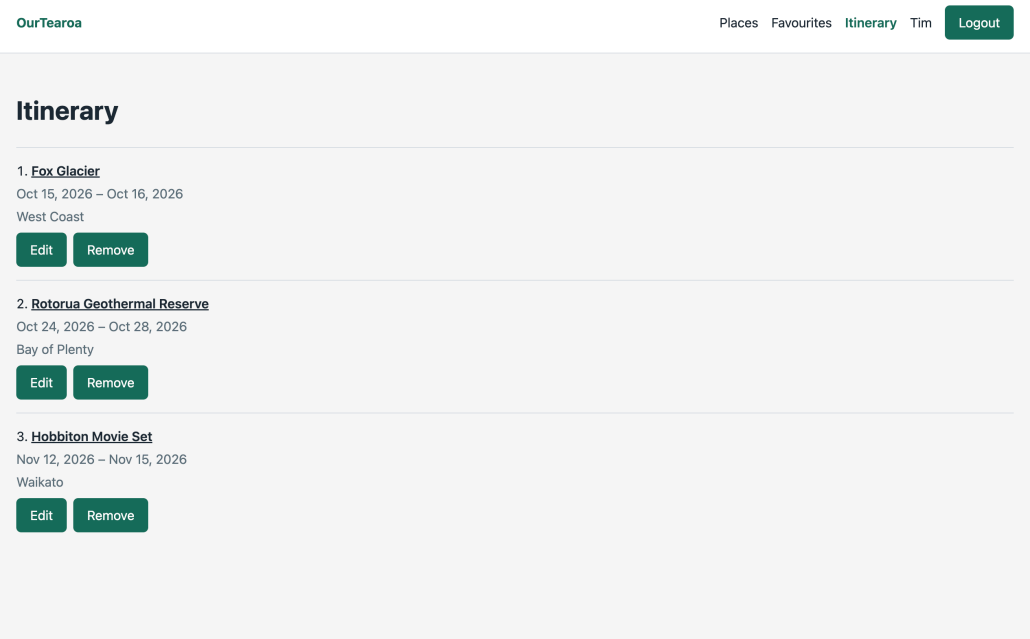


Figure 24: Itinerary ordered by start date

Step 5: Rate and review a place. In the *Reviews* section of the detail panel, select 1 to 5 stars, write an optional comment and select *Post review*. Your own reviews show *Edit* and *Delete*; other users' reviews cannot be changed.

Step 6: Add a photo. Signed-in users can select *Upload a photo* in the detail panel and choose a JPEG, PNG or WebP file of up to 5 MB. The image is uploaded directly to cloud storage (S3) and replaces the current photo.

Step 7: Manage the account. Select your name in the header to open *Profile*, which shows name, email and *Member since*. *Delete account* asks for confirmation and then permanently removes the account together with its reviews, favourites and itinerary entries.

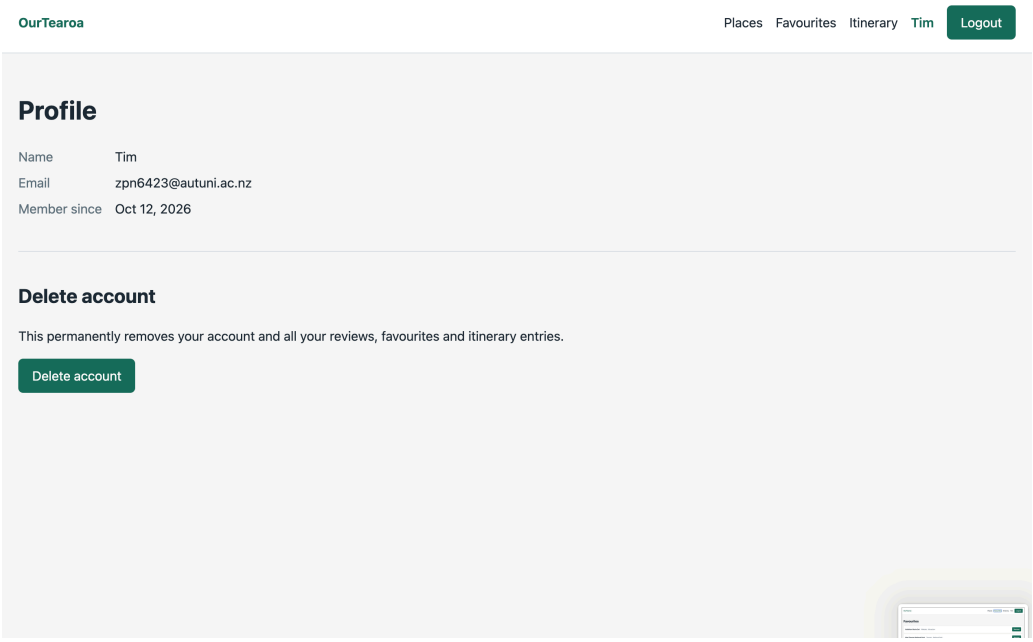


Figure 25: Profile page with account deletion

9. RESTful API List

9.1. API Overview & Evaluation Details

All endpoints are under /api and speak JSON. Responses are 200/201 on success and a JSON { "error": "..." } body on failure. Base URL: https://ourtearoa.duckdns.org/api (local: http://localhost:3000/api). "Auth" means an Authorization: Bearer <JWT> header from POST /auth/login or POST /auth/register. Another user's resource returns 404, not 403. GET /api/health returns {"status":"UP","database":"UP"}. scripts/api-smoke-test.sh [BASE_URL] exercises them without the client. There are 25 endpoints, 8 of them POST.

9.2. Endpoints Specification Table

Method	Path	Auth	Status	Notes
GET	/health	No	200	API and database status
POST	/auth/register	No	201	email, name, password (min 8); 409 if email exists
POST	/auth/login	No	200	returns token and user; 401 on bad credentials
GET	/users/me	Bearer	200	own profile including email
GET	/users/:id	Bearer	200	id, name, createdAt only

Method	Path	Auth	Status	Notes
DELETE	/users/me	Bearer	204	deletes own account and, via cascade, its reviews, favorites and itinerary
GET	/favorites	Bearer	200	own favorites with place
POST	/favorites	Bearer	201	placeId; 409 if already a favorite
DELETE	/favorites/:id	Bearer	204	own favorites only
GET	/itinerary	Bearer	200	own entries ordered by start date
POST	/itinerary	Bearer	201	placeId, startDate, endDate, note?; 400 if end before start
PUT	/itinerary/:id	Bearer	200	partial update
DELETE	/itinerary/:id	Bearer	204	own entries only
GET	/places/:id/reviews	No	200	reviews with author name
GET	/reviews/:id	No	200	
POST	/reviews	Bearer	201	placeId, rating (1-5), comment?
PUT	/reviews/:id	Bearer	200	own reviews only
DELETE	/reviews/:id	Bearer	204	own reviews only
GET	/places	No	200	optional region (exact, case-insensitive) and q (name contains, case-insensitive); sorted by name
GET	/places/:id	No	200	404 if unknown
POST	/places	Bearer	201	name (max 200), region (max 100), description?, latitude? (-90..90), longitude? (-180..180), type?
PUT	/places/:id	Bearer	200	partial update with the same validation
DELETE	/places/:id	Bearer	204	cascades to its reviews, favourites and itinerary entries
POST	/places/:id/image-upload-url	Bearer	201	contentType (jpeg/png/webp); presigned S3 POST, key, imageUrl; 503 if S3 is not configured
POST	/places/:id/image	Bearer	200	key from the upload step; checks the object exists and saves imageUrl

9.3. API Test and Smoke Test Evidence

The API was tested with scripts rather than Postman, so that the tests are repeatable and stored in the repository. scripts/api-smoke-test.sh calls every endpoint against a given base URL with curl: it registers a temporary user, logs in to obtain a JWT, exercises places, favourites, itinerary and reviews including error cases (400, 401, 404, 409), and finally deletes the user again. The run against the public cloud URL passed all 45 checks (docs/evidence/api-smoke-test-public.log):

```
~ ❯ curl https://ourtearoa.duckdns.org/api/health
{"status":"UP","timestamp":"2026-10-12T01:27:57.637Z","database":"UP","latencyMs":53.95,"reason":null}
```

Figure 26: GET /api/health on the public URL returning "status":"UP" and "database":"UP"

```
Run: 2026-10-11T22:06:06Z
Command: scripts/api-smoke-test.sh https://ourtearoa.duckdns.org/api

PASS  GET  /health                                  200
PASS  POST /auth/register                           201
PASS  POST /auth/register (duplicate email)         409
PASS  POST /auth/register (invalid input)           400
PASS  POST /auth/login (wrong password)             401
PASS  POST /auth/login                              200
...
PASS  POST /itinerary (end before start)            400
PASS  GET  /itinerary                               200
PASS  PUT  /itinerary/:id                           200
PASS  DELETE /itinerary/:id                         204
PASS  POST /reviews (rating out of range)           400
PASS  POST /reviews                                 201
...
PASS  DELETE /users/me (cleanup)                    204
PASS  GET  /users/me (after delete)                 404

Result: 45 passed, 0 failed, 0 skipped
```

Listing 1: Excerpt of the smoke test run against the public API

The S3 image upload was tested end to end in the same way (docs/evidence/s3-upload-public.log). The browser-side upload goes straight to S3 without an Authorization header, and S3 itself rejects a file over 5 MB because of the signed policy:

```
POST /places/:id/image-upload-url                   201
  upload url: https://ourtearoa-storage.s3.ap-southeast-2.amazonaws.com/
POST <S3 bucket> multipart upload, no Authorization header   204
POST <S3 bucket> upload over 5 MB (rejected by S3 policy)    400
POST /places/:id/image (confirm)                    200
HEAD imageUrl (public read from S3)                 200 server=AmazonS3 type=image/png
```

Listing 2: Excerpt of the S3 presigned upload test against the public API

10. Summary

OurTearoa is a working three-tier cloud application: an Angular client, an Express REST API on EC2, and managed PostgreSQL on RDS, with images in S3. It is publicly reachable over HTTPS and redeployed automatically on every merge to main.

10.1. Evaluation Against Brief

Option B requirement	How OurTearoa meets it
Three tiers	Angular SPA (client), Express 5 API (application), RDS PostgreSQL (data) (Section 7)
Client and API decoupled by REST	The client only talks to /api over JSON; the API is usable without the client (Section 9.1)
At least 10 REST services, at least 3 POST	25 endpoints, 8 of them POST
Cloud database, not installed on the VM	AWS RDS for PostgreSQL 16 over verified TLS (Section 5.4)
Assets in cloud object storage	Place images in S3 via presigned direct upload (Section 5.5)
Public URLs for client and API	Client: https://ourtearoa.duckdns.org API: https://ourtearoa.duckdns.org/api
Git collaboration without history rewriting	Feature branches, reviewed pull requests linked to issues, no force-push

All functional requirements are met except the contributor stories US-14 and US-15, which exist in the API but have no screen and no ownership model (Appendix B). Beyond the brief, the deployment is automated with tests before every deploy, and the system is evaluated for security, availability, backup, scalability, performance and cost in Section 7. Some security hardening was planned but not completed before submission: the RDS instance is still publicly accessible for development, the team shares one AWS administrator account, and there is no monitoring yet. These are stated as open points in Section 7.1 and Section 7.7.

10.2. Critical Engineering Reflections & Lessons Learned

Working within 1 GB of memory

The t3.micro was chosen for cost, and that choice shaped more of the design than expected. Building the Angular client needs more memory than the instance has, so the production build was moved to GitHub Actions and only the compiled files are copied to the server. A swap file protects the running API from the OOM killer. The same reasoning led to uploading images directly from the browser to S3 instead of through the API. The lesson: on a small instance, keep build work and bulk data off the server and give it only the job of serving requests.

Deployment needs to be repeatable, not just automated

Several problems appeared only in the pipeline: wrong secret interpolation in the deploy step, the backend deployed from the wrong directory, two overlapping runs deleting each other's

node_modules, and SELinux blocking Nginx from reading newly copied files. Each fix (a concurrency group, restoring the SELinux context, pinned Prisma via npx –no, migrate deploy only) made the deploy more predictable. Running the client tests before every deploy turned the pipeline from a copy step into a quality gate.

Security is layered, and some layers are still open.

TLS is used on every hop: Let's Encrypt at Nginx, sslmode=verify-full with the AWS CA bundle to RDS, and HTTPS to S3. The EC2 instance uses an IAM role, so no AWS keys exist on the server, and the API checks ownership on every user-owned row. A distinct 503 reason in the health check (timeout versus unreachable) made it quick to tell a security group problem from a credential problem. What we would do differently: make RDS private from the start and give each team member their own IAM user, instead of opening developer IP addresses and sharing one administrator.

Teamwork

Splitting work into GitHub issues and small pull requests with review kept both members informed and the history readable. The bottleneck was AWS console access, held by one member; documenting the setup in docs/ reduced but did not remove that dependency.

Production enhancements

For real traffic, the next steps are those in Section 7.7: monitoring and a budget alert first, then serving the client and images through CloudFront, and finally an Application Load Balancer with an Auto Scaling Group across two Availability Zones and RDS Multi-AZ. Because the API is stateless and all data lives in RDS and S3, these steps need little or no change to the application code. A cache such as ElastiCache (Redis) for the public place catalogue could be added if database reads become the bottleneck.

References

[1] Google LLC, "Angular Documentation: The Modern Web Developer's Platform." Accessed: Oct. 12, 2026. [Online]. Available: https://angular.dev/

[2] Amazon Web Services, "Amazon Relational Database Service (Amazon RDS) for PostgreSQL User Guide." Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/

[3] Amazon Web Services, "Amazon Elastic Compute Cloud (Amazon EC2) Documentation." Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/ec2/

[4] Amazon Web Services, "Amazon Simple Storage Service (Amazon S3) Documentation." Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/s3/

[5] Internet Security Research Group, "Let's Encrypt: A Free, Automated, and Open Certificate Authority." Accessed: Oct. 12, 2026. [Online]. Available: https://letsencrypt.org/docs/

[6] Duck DNS, "Duck DNS: Free Dynamic DNS Hosted on Amazon VPC." Accessed: Oct. 12, 2026. [Online]. Available: https://www.duckdns.org/

[7] Amazon Web Services, “Creating a Presigned POST Policy in Amazon S3.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html

[8] Amazon Web Services, “Use IMDSv2 to Secure Amazon EC2 Instance Metadata.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html

[9] GitHub, Inc., “GitHub Actions Documentation: Continuous Integration and Continuous Delivery (CI/CD).” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.github.com/en/actions

[10] Amazon Web Services, “Amazon CloudFront Developer Guide.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/

Appendix A: AI Usage Declaration form

Please list the tasks in your assignment where AI was used.

- Coding assistance for the Angular client and the Express API (for example feature components, form validation, controllers and unit tests).
- Troubleshooting the deployment: GitHub Actions workflow, Nginx, PM2 and the S3 upload flow.
- Drafting and restructuring technical documentation (DESIGN.md, setup guides) and parts of this report.
- Wording of GitHub issues and pull request descriptions, and reviewing our own changes before merging.

For each task, explain why the AI tool was used (e.g., idea generation, summarization, coding, etc.).

- **Coding:** to speed up routine implementation so that more time was left for the cloud architecture, which is the focus of the course.
- **Debugging:** to interpret error messages and logs from the pipeline and the server faster.
- **Writing and summarization:** to turn our design decisions, notes and evidence into structured text, and to check the report for consistency.
- **Review:** as a second reviewer that points out bugs, missing edge cases and inconsistencies.

Please state the AI tools used.

- Abacus.AI Agent (AI coding assistant in the IDE)
- GitHub Copilot coding agent (two pull requests fixing the deploy workflow, September 2026)

List the main prompts (less than 5) or queries entered into the AI tool

1. "Implement this GitHub issue in the Angular client against the existing REST endpoints, including loading, empty and error states and unit tests."
2. "The deploy workflow fails at this step; here is the log. Find the cause and propose a fix."
3. "Review my changes for bugs and security issues before I open the pull request."
4. "Transfer the content of DESIGN.md into the report and check it against the current code and AWS setup."

Provide a summary or description of the AI-generated output. (Optional: attach output in an appendix.)

Code changes on feature branches (components, controllers, tests and workflow fixes), explanations of errors with proposed fixes, review findings, and draft text for documentation, issues and pull requests. The resulting code and text can be traced in the Git history and the pull requests of the repository.

Please describe how the AI output was used, edited, or transformed.

The architecture, the choice of AWS services, the AWS configuration and all trade-offs were decided by the team; the AWS resources were set up by hand in the console. AI output was treated as a draft: code changes went through pull requests that a team member reviewed

before merging, and was checked by the client test suite, the smoke test and the live health check. Documentation and report text were checked against the actual code, the AWS console and the evidence in docs/evidence/, and corrected, shortened or rewritten where it was wrong, too general or not our wording. Suggestions that did not fit the project were rejected. We take full responsibility for the submitted work.

Student name(s):	**Signature(s):**	**Date:**
Mathias Jørgensen (26344066)		16 October 2026
Tim Schiller (26347002)		16 October 2026

Appendix B: User Stories and Acceptance Criteria

These are the core requirements of the system; every other design decision in this report traces back to one of the stories below.

Visitor

ID	User story	Acceptance criteria
US-01	As a visitor, I want to browse a list of New Zealand places so that I can discover destinations without an account.	GET /places is public; /places lists every place as a card without login. Done.
US-02	As a visitor, I want to filter and search places by region so that I can narrow down to the part of the country I care about.	Region dropdown and name search on /places, backed by ?region= (exact, case-insensitive) and ?q= (name contains). Done.
US-03	As a visitor, I want to open a place detail page showing name, region, description, photo and map coordinates so that I can decide whether to visit.	/places/:id shows name, region, type, description, the S3 photo when imageUrl is set and a Leaflet map when both coordinates are set. Done.
US-04	As a visitor, I want to read reviews and ratings left by other travellers so that I can judge a place from other people's experiences.	GET /places/:id/reviews is public; the detail panel lists rating, comment and author name. Done.

Registered User (Traveller)

ID	User story	Acceptance criteria
US-05	As a visitor, I want to register an account so that I can save favourites and plan an itinerary.	POST /auth/register with email, name (max 100) and password (min 8); 409 on a duplicate email; the user is signed in afterwards. Done.
US-06	As a returning user, I want to log in so that I can access my saved data on any device.	POST /auth/login returns a JWT valid for 7 days; wrong credentials give 401; data is server-side, so it is the same on any device. Done.
US-07	As a registered user, I want to favourite a place so that I can build a shortlist.	Heart toggle on the detail panel and on every card; POST /favorites, 409 on a duplicate. Done.
US-08	As a registered user, I want to view and remove my favourites so that my shortlist stays current.	/favorites lists saved places with remove (DELETE /favorites/:id). Done.

ID	User story	Acceptance criteria
US-09	As a registered user, I want to write a review and star rating for a place I have visited so that I can share my experience.	Review form with a 1-5 rating and optional comment (POST /reviews, 400 outside 1-5). Done.
US-10	As a registered user, I want to edit or delete my own review so that I can correct it later.	Edit and delete are offered only on the user's own reviews; the API answers 404 for anyone else's. Done.
US-11	As a registered user, I want to add places to an itinerary with start and end dates so that I can organise a trip.	"Plan a visit" on the detail panel with start date, end date and note; 400 if the end is before the start. Done.
US-12	As a registered user, I want to view my itinerary ordered by date so that I can plan the sequence of my trip.	/itinerary lists entries ordered by start date (GET /itinerary, indexed on user and start date). Done.
US-13	As a registered user, I want to edit or remove itinerary entries so that my plan can change as my trip does.	Edit and remove on /itinerary (PUT/DELETE /itinerary/:id, own entries only). Done.

Contributor (Later-Stage Feature)

ID	User story	Acceptance criteria
US-14	As a contributor, I want to add a new place with its details and image so that the catalogue grows beyond the seed data.	API only: POST /places and the S3 image upload exist; no "add place" screen. Partial.
US-15	As a contributor, I want to edit or remove a place I have added so that catalogue data stays accurate.	API only: PUT/DELETE /places/:id exist, but places have no owner field, so any signed-in user can change any place. Partial.

US-16 (contributor image upload to cloud storage) was removed as a separate story. Place images are covered by the S3 presigned direct upload (issues #71, #73 and #87): any signed-in user can attach a photo on the place detail panel. An earlier plan to store images on the EC2 disk was dropped: it would put upload traffic and storage on the 1 GB t3.micro, and every image would be lost if the instance were replaced.

Evaluator / Operations

ID	User story	Acceptance criteria
US-17	As an evaluator, I want a documented set of RESTful endpoints that I can call directly, so that the API can be tested independently of the UI.	All endpoints listed in the API section of this report; scripts/api-smoke-test.sh calls them without the UI (45 checks). Done.
US-18	As an evaluator, I want a health endpoint that reports both the API and the database connection, so that I can confirm the whole stack is live.	GET /api/health runs SELECT 1 on RDS and returns 200 with {"status":"UP","database":"UP"} and latencyMs, or 503 with a reason. Done.
US-19	As a user on a slow or unreliable connection, I want clear loading and error states so that the interface never appears frozen.	Every data screen has loading, empty and error states, with a retry button on errors. Done.
US-20	As a user, I want the app served over HTTPS on a memorable domain so that it is trustworthy and easy to reach.	https://ourtearoa.duckdns.org with a Let's Encrypt certificate; HTTP answers 301 to HTTPS. Done.